Module 7 Week A — Reading: Transformer Architectures & Fine-Tuning

Everything in this reading is fair game for readiness checks, drills, and labs. A learner who reads this carefully should be able to complete the Week A drill and lab without attending the concept session.

What success looks like: After this reading, you can explain the three transformer architectural families (encoder, decoder, encoder–decoder) and identify which fits a given Natural Language Processing (NLP) task; articulate what fine-tuning does and why it works; prepare a labeled text dataset using the Hugging Face `datasets` library; configure and run a fine-tuning loop with `transformers.Trainer`; and evaluate a fine-tuned classifier with appropriate metrics.

In This Reading

1. Three Architectures, Three Jobs
2. The Fine-Tuning Paradigm
3. Task-Specific Heads
4. BERT Family Models — When to Pick Which
5. Decoder Fine-Tuning at a Glance
6. Preparing a Dataset for Fine-Tuning
7. The Trainer API
8. Tokenization Mechanics for Training
9. Evaluation for Sequence Classification
10. Compute Reality and Engineering Trade-offs

- Key Terms
- Summary
- What's Next

---

1. Three Architectures, Three Jobs

In Module 6 you used pre-trained transformer models (DistilBERT) to extract contextual embeddings — vectors that represent the meaning of tokens in context. You loaded a model with `AutoModel.from_pretrained(...)`, ran text through it, and pulled hidden states out. That was *consumption* — using the model as-is. This module is about adaptation — taking a pre-trained model and updating its weights so it does a specific task well on your data. That process is called **fine-tuning**, and it is the dominant paradigm of applied modern NLP.

Before we fine-tune anything, we need a stable mental map of the transformer model space. Modern transformer models fall into three architectural families, each suited to a different shape of problem. The table below is a compact reference; the prose that follows walks through each family in turn.

Family	Attention pattern	Canonical models	Canonical tasks
Encoder	Bidirectional	BERT, DistilBERT, RoBERTa, DeBERTa	Sequence classification, token classification, sentence pair, extractive QA
Decoder	Causal (left-to-right)	GPT-2/3/4, Llama, Mistral, Qwen	Generation, code completion, instruction following, in-context learning
Encoder–decoder	Bidirectional encoder + causal decoder	T5, BART, mT5, Flan-T5, Whisper	Translation, summarization, generative QA, speech-to-text

Encoder models

An **encoder** transformer takes a sequence of tokens as input and produces a sequence of contextual representations (hidden states) — one per input token. It reads the input bidirectionally: every token's representation can attend to every other token in the sequence. This makes encoders excellent at *understanding* an input.

Canonical encoder models: **BERT, DistilBERT, RoBERTa, DeBERTa.**

Canonical encoder tasks:

- Sequence classification (sentiment, topic, intent)
- Token classification (Named Entity Recognition, part-of-speech tagging)
- Sentence pair tasks (entailment, similarity)
- Extractive question answering (find the span that answers the question)

The Module 6 NER pipeline used an encoder under the hood. The Lab in this Week A fine-tunes an encoder for sentiment classification.

Decoder models

A **decoder** transformer also takes a sequence of tokens, but it produces output one token at a time, generating the next token based only on what came before (causal / left-to-right attention). It is built for *generation*.

Canonical decoder models: **GPT-2, GPT-3, GPT-4, Llama, Mistral, Qwen.**

Canonical decoder tasks:

- Open-ended text generation
- Code completion
- Instruction following / chat
- In-context learning (few-shot prompting)

When you use ChatGPT, Claude, or any chat-style assistant, you are interacting with a decoder. Fine-tuning a large decoder is computationally expensive (we will return to this in Section 5).

Encoder–decoder models

An **encoder–decoder** transformer combines both: an encoder reads the input, and a decoder generates a different output sequence based on the encoder's output. This is the right architecture when input and output are *both sequences but different sequences*.

Canonical encoder–decoder models: **T5, BART, mT5, Flan-T5.**

Canonical encoder–decoder tasks:

- Translation (input language → output language)
- Summarization (long document → short summary)
- Generative question answering (question + context → answer)
- Speech-to-text (audio → text — the Whisper model is encoder–decoder)

Week B will use a pre-trained encoder–decoder model for summarization.

How to remember the difference

A useful shortcut:

- Encoder = "understand the input." Output is a fixed-shape representation, not new text.
- Decoder = "continue the input." Output is more text, generated one token at a time.
- Encoder–decoder = "transform input into different output." Output is text, but in a different shape, language, or genre than the input.

When someone says "transformer," they could mean any of these three. The architectural family matters because it determines the natural shape of the task you can do with the model and the kind of head you attach for fine-tuning.

---

2. The Fine-Tuning Paradigm

Modern NLP rests on a two-stage paradigm: **pre-training** followed by **fine-tuning**.

Pre-training

A model is pre-trained on a massive, general corpus (often hundreds of billions of tokens of internet text and books) using a self-supervised objective. For encoders like BERT, the objective is **masked language modeling**: the model sees sentences with some tokens hidden ("The cat sat on the [MASK]") and must predict the missing tokens. For decoders like GPT, the objective is **causal language modeling**: predict the next token given everything that came before.

Pre-training is enormously expensive — millions of GPU-hours, millions of dollars in compute. Pre-training is something a few large organizations do; the rest of us consume the result.

What pre-training produces is a model with rich, general language understanding baked into its weights — grammar, syntax, broad semantic associations, basic world knowledge. The model has not yet been told *what task to do*, but it has learned what language is.

Fine-tuning

Fine-tuning takes a pre-trained model and continues training it — but on a much smaller, task-specific labeled dataset, with a task-specific objective. For sequence classification (the Lab task), the dataset is labeled sentences and the objective is to predict the correct label. The training loop runs for a few epochs (often 2–4), updating the model's weights so it gets better at the specific task.

The crucial difference from training-from-scratch:

	From scratch	Fine-tuning
Starting weights	Random	Pre-trained (rich language baked in)
Dataset size needed	Hundreds of thousands to millions of labeled examples	Hundreds to a few thousand examples
Training time	Days to weeks	Minutes to hours
Cost	High	Low
Final performance	Usually worse	Usually better

This counterintuitive result — that starting from a pre-trained model and adjusting it produces a better domain classifier than training a domain-specific model from scratch — is the central engineering insight of modern NLP. The intuition is **transfer learning**: language structure is reusable. A model that has already learned what nouns and verbs are and how sentences are organized has done the hard part. Specializing it to your domain task is comparatively cheap.

What gets updated during fine-tuning?

By default, *all* model weights update during fine-tuning, including the entire pre-trained backbone. This is sometimes called "full fine-tuning."

There is a related family of techniques called **parameter-efficient fine-tuning** (PEFT) — including LoRA, adapters, and prefix tuning — where most of the backbone is frozen and only a small set of additional parameters update. PEFT is increasingly common for very large models because it uses less memory. We mention this for vocabulary, but **PEFT is not part of this module**. The Lab does full fine-tuning.

A common misconception is that fine-tuning freezes the backbone and only trains a "head" on top. That is **not** what default fine-tuning does. Fine-tuning updates everything. The head is a separate concept — covered in Section 3.

---

3. Task-Specific Heads

A pre-trained transformer is just a backbone. The backbone produces hidden states (one per input token); it does not produce predictions in the shape of any specific task. To use the backbone for a task, we add a small **task-specific head** on top.

What a head looks like

A head is typically a tiny neural network — often just one or two linear layers — that takes hidden states from the backbone and produces task-shaped outputs.

For **sequence classification** (e.g., sentiment), the head:

1. Takes the hidden state of the special `[CLS]` token — the first token, designed to summarize the whole input. This is the BERT/DistilBERT convention; some other encoder families use a different special token (e.g., RoBERTa uses `<s>`). The Hugging Face tokenizer prepends the right one for the model you load, so you don't pick this manually.
2. Passes it through a linear layer to produce one logit per class
3. Applies softmax (during inference) to get class probabilities

For **token classification** (e.g., NER), the head produces one logit per class for *every* token, not just `[CLS]`.

For **extractive question answering**, the head produces two logits per token — one for "is this the start of the answer span?" and one for "is this the end?".

The head's output shape determines what the model can predict. In the Hugging Face library, the right head is selected by which `AutoModelFor...` class you load:

Class	Head	Task
`AutoModelForSequenceClassification`	`[CLS]` → linear → logits over `num_labels` classes	Classification, sentiment
`AutoModelForTokenClassification`	per-token → linear → logits	NER, POS tagging
`AutoModelForQuestionAnswering`	per-token → 2 linears → start/end logits	Extractive QA
`AutoModelForCausalLM`	per-token → linear → logits over vocabulary	Decoder generation
`AutoModelForSeq2SeqLM`	encoder–decoder generation	Translation, summarization

When you load `AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=3)`, two things happen:

1. The pre-trained backbone weights are loaded from the public DistilBERT checkpoint.
2. A new, **randomly initialized** classification head with output dimension 3 is attached.

The randomness of the head is important. Before training, the head produces near-random predictions (for 3 classes, ~33% accuracy). The head's job during fine-tuning is to learn — fast — what features in the backbone's hidden states correspond to each class. Because the head is small (often just a few thousand parameters compared to the backbone's tens of millions), it converges quickly.

Why running inference on an untrained model gives garbage

A common moment of confusion: a learner loads a freshly initialized `AutoModelForSequenceClassification`, runs it on a sentence, and sees random class predictions. They assume something is broken. Nothing is broken — the head is randomly initialized and has never seen training data. After fine-tuning, predictions become meaningful. **This is not a bug; it is the expected pre-training state.**

---

4. BERT Family Models — When to Pick Which

For this Week A, you will fine-tune **DistilBERT**. Several other BERT-family models are commonly available — knowing the differences helps you make informed choices in the future.

DistilBERT

A "distilled" version of BERT-base. Created by training a smaller model to mimic the outputs of BERT-base. About 66 million parameters — roughly 40% smaller than BERT-base. About 60% faster at inference. Retains roughly 95% of BERT-base's downstream task performance.

Why we picked it for this Lab: CPU fine-tuning is feasible at our dataset scale. On a typical laptop, fine-tuning DistilBERT on ~7,500 app-review examples for 2 epochs takes ~60–90 minutes. BERT-base would take roughly twice as long. RoBERTa-large would take many hours and likely exceed available memory.

BERT (BERT-base, BERT-large)

The original 2018 model that started the "fine-tune a pre-trained transformer" revolution. BERT-base has ~110M parameters. BERT-large has ~340M. Still widely used. Use it when you can afford the additional compute and want slightly better quality than DistilBERT.

RoBERTa

"Robustly Optimized BERT Approach." Same architecture as BERT, but trained with several recipe improvements: more training data, longer training, dynamic masking (the masked tokens change every epoch), and removal of the Next Sentence Prediction objective. RoBERTa-base (~125M parameters) typically beats BERT-base on benchmarks by a few points. Use it when you want a stronger drop-in encoder and have GPU access.

In this Lab, RoBERTa is **conceptual only** — we describe it but do not fine-tune it. If you have GPU access after the course and want to swap, replacing `"distilbert-base-uncased"` with `"roberta-base"` and adjusting the tokenizer is usually enough. The training code does not need to change.

DeBERTa, ALBERT, ELECTRA

Later generation BERT-family models with various architectural and training improvements. DeBERTa is currently among the strongest encoders on classification benchmarks. ALBERT shares parameters across layers to reduce memory. ELECTRA uses a different pre-training objective (replaced-token detection) that improves sample efficiency.

You don't need to memorize these. The point is: the BERT family contains many options, all interchangeable architecturally for sequence classification, differing in size, speed, and benchmark performance.

---

5. Decoder Fine-Tuning at a Glance

Fine-tuning a decoder model (GPT family) is a different beast. We cover it conceptually in this section so you understand the boundary between this Lab (encoder fine-tuning) and what you would do for a decoder task.

What changes for decoders

A decoder fine-tune typically uses **causal language modeling** (CLM) as the objective: given the current token, predict the next token. The training data is text — not labels and texts, just text — and the loss is computed over every position in the sequence. Common decoder fine-tuning regimes:

- **Continued pre-training** on domain text — the model continues learning the language, but on your domain (legal, medical, financial). No labels needed.
- **Instruction tuning** — fine-tuning on (instruction, response) pairs to make a base decoder model follow instructions. This is what turned GPT-3 into ChatGPT.
- **Reinforcement Learning from Human Feedback (RLHF)** — a further refinement on top of instruction tuning, using human preference data. This is what makes a decoder model feel polished and aligned.

Why you cannot fine-tune GPT on a CPU

Decoder fine-tuning typically requires:

- **More memory** — decoders are usually larger (modern open decoders run from 1B to 70B+ parameters; encoders for fine-tuning are typically 60M–400M).
- **Generation at evaluation time** — testing the model's output requires generating tokens one-at-a-time, which is slow on CPU.
- **Longer sequences** — instruction tuning typically uses sequences of 512–2048 tokens, vs. the 128 we use for sentiment classification. Longer sequences quadratically increase attention compute.

Fine-tuning a 7-billion-parameter decoder on a single laptop CPU is essentially infeasible — it would take weeks per epoch, if it fit in memory at all.

What this means for the course

In this module, **decoder fine-tuning is concept-only** — you should be able to describe the differences but you will not fine-tune one. The course favors techniques that work on the cohort's actual hardware. When you have GPU access in your professional life, the same fine-tuning concepts you learn this week (tokenization with padding, `Trainer`, `compute_metrics`, the train/val/test split) all transfer directly to decoder fine-tuning.

---

6. Preparing a Dataset for Fine-Tuning

Most of fine-tuning is data preparation. The actual `trainer.train()` call is one line; getting the dataset into the right shape is many.

The Hugging Face `datasets` library is the standard tool. It provides a uniform interface to public datasets (via `load_dataset`) and to your own custom data (via `Dataset.from_pandas`, `Dataset.from_dict`, or `load_dataset("csv", ...)`).

Loading a CSV that ships with your repo

The lab ships its training data as a CSV (`data/app_reviews_train.csv`, ~7,500 reviews). Two equivalent paths to a `Dataset`:

```python
import pandas as pd
from datasets import Dataset

df = pd.read_csv("data/app_reviews_train.csv")
ds = Dataset.from_pandas(df, preserve_index=False)
```

Or load directly with the `csv` builder:

```python
from datasets import load_dataset

ds = load_dataset("csv", data_files="data/app_reviews_train.csv")["train"]
```

Either approach returns a `Dataset`. We then call `.train_test_split(test_size=0.2, seed=42)` to produce a `DatasetDict` with `train` and `test` keys.

Loading from a CSV

```python
from datasets import Dataset
import pandas as pd

df = pd.read_csv("my_data.csv")
ds = Dataset.from_pandas(df)
```

Splitting a dataset

```python
ds_split = ds.train_test_split(test_size=0.2, seed=42)
# Returns a DatasetDict with "train" and "test" keys
```

Always pin the seed. Training is non-deterministic enough already; the split should be reproducible.

Tokenizing the dataset

Fine-tuning requires tokenized text — token IDs and attention masks, not raw strings. The standard pattern is to use `Dataset.map` with `batched=True`:

```python
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def tokenize_fn(batch):
    return tokenizer(batch["sentence"], truncation=True, max_length=128)

tokenized = ds.map(tokenize_fn, batched=True)
```

Three things to notice:

- **`batched=True`** — `tokenize_fn` receives a batch of examples (a dict of lists) instead of one example at a time. It is significantly faster.
- **`truncation=True, max_length=128`** — sentences longer than 128 tokens are truncated to 128. This is required: the model has a maximum input length, and longer sequences must be cut.
- **No `padding` argument** — we pad later, dynamically per batch, via `DataCollatorWithPadding` (Section 8). Padding here would pad every example to 128, which wastes compute on short sentences.

After `map`, the dataset has new columns: `input_ids` and `attention_mask`. The original `sentence` column is still there but unused by the model.

Setting the format

For the model to consume the dataset, it must be in PyTorch tensor format:

```python
tokenized.set_format("torch", columns=["input_ids", "attention_mask", "label"])
```

This tells the dataset to return PyTorch tensors when iterated, and to only return the columns the model needs. The `label` column is the integer class label.

A common error: forgetting to include `label` in the columns list. The model needs to compare its predictions against `label` to compute the loss. Without it, training silently produces garbage.

---

7. The Trainer API

The `transformers.Trainer` class wraps the entire training loop. You configure it with `TrainingArguments` and a model, datasets, tokenizer, and metric function — and `trainer.train()` runs the full loop, including checkpointing and evaluation.

TrainingArguments

```python
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="model",
    learning_rate=5e-5,
    num_train_epochs=2,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    eval_strategy="epoch",
    save_strategy="epoch",
    seed=42,
    logging_steps=50,
)
```

Key arguments:

- **`output_dir`** — where checkpoints and logs are saved.
- **`learning_rate=5e-5`** — the standard starting learning rate for BERT-family fine-tuning. An order of magnitude lower than typical scratch-training rates because the pre-trained backbone weights should change slowly.
- **`num_train_epochs=2`** — passes through the full training data. For BERT-family fine-tuning on small datasets, 2–4 epochs is the typical range. More epochs risk overfitting.
- **`per_device_train_batch_size=8`** — small batch size to keep memory low on CPU. For DistilBERT at `max_length=128`, batch size 8 fits comfortably in 4–6 GB of RAM.
- **`eval_strategy="epoch"`, `save_strategy="epoch"`** — evaluate and save after every epoch. Lets you watch metrics improve.
- **`seed=42`** — pin the random seed for reproducibility.

Trainer

```python
from transformers import Trainer, AutoModelForSequenceClassification, DataCollato

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=3
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer=tokenizer),
    compute_metrics=compute_metrics,
)

trainer.train()
```

`Trainer` handles:

- Looping over the training data
- Computing the loss and gradient updates
- Periodic evaluation against `eval_dataset` using `compute_metrics`
- Saving checkpoints to `output_dir` per `save_strategy`
- Logging metrics

After `trainer.train()` completes, the model has been fine-tuned. To save the final state:

```python
trainer.save_model("model")
tokenizer.save_pretrained("model")
```

Note that `save_model` saves the model weights and config but **not the tokenizer**. Always save both. When you load the model later, you will need both to do inference.

Pushing to Hugging Face Hub

For Lab 7A, the final step pushes the fine-tuned model to your free Hugging Face Hub account so the Integration task can load it without you having to commit a 265 MB checkpoint to GitHub:

```python
trainer.push_to_hub("m7-app-review-sentiment")
tokenizer.push_to_hub("m7-app-review-sentiment")
```

You first need to authenticate once with `huggingface-cli login` (using a token from your HF account settings — separate from your GitHub PAT). The Lab guide walks through the setup.

---

8. Tokenization Mechanics for Training

Section 6 introduced tokenization at a high level. Three mechanical points matter for fine-tuning specifically.

Padding and the attention mask

Transformer models process fixed-shape tensors. When sentences in a batch have different lengths, shorter sentences are padded with a special pad token to match the longest sentence in the batch.

Padding poses a problem: the model would otherwise treat pad tokens as real tokens during attention, producing nonsense. The **attention mask** solves this: a binary mask, 1 for real tokens, 0 for pad tokens. The model uses the mask to ignore pad positions in attention computation.

The tokenizer produces both `input_ids` and `attention_mask`. Both must be present in the dataset for the model to train correctly. If you ever see training results that look terrible, one debugging step is to print a batch and verify the attention mask is what you expect.

DataCollatorWithPadding

In Section 6 we tokenized without padding (`padding=False`). At training time, we pad dynamically per-batch via `DataCollatorWithPadding`. The collator looks at each batch as it is assembled and pads to the longest sequence in that batch.

Why dynamic padding instead of padding everything to 128 up front?

- **Compute savings.** A batch where all sentences are short (say, 30 tokens) processes much faster than the same batch padded to 128. Over thousands of batches, this adds up significantly.
- **Memory savings.** Smaller batches mean lower peak memory, which matters on CPU.

You wire it up by passing `data_collator=DataCollatorWithPadding(tokenizer=tokenizer)` to `Trainer`.

Truncation

Sentences longer than `max_length` must be truncated. For BERT-family models, the architectural maximum is 512 tokens, but for fine-tuning on consumer hardware, smaller values (128, 256) are typical. Set `max_length=128` for sentence-level tasks like sentiment.

What happens with the truncated content depends on the truncation strategy. The default (`truncation=True`) truncates from the end ("longest_first"). For most short-text classification tasks, this is fine. For tasks where the meaningful content might be at the end (rare in classification), other strategies exist.

---

9. Evaluation for Sequence Classification

Once the model is trained, you evaluate it on the held-out test split. For sequence classification, the standard metrics are:

Accuracy

The fraction of test examples the model classifies correctly. Simple, intuitive, and **misleading when classes are imbalanced**. If 90% of your test set is class A, predicting class A for everything yields 90% accuracy — even though the model learned nothing.

Precision, Recall, F1

For each class, three metrics:

- **Precision** — of the examples the model predicted as this class, what fraction was actually this class? (High precision = few false positives.)
- **Recall** — of the examples that actually were this class, what fraction did the model catch? (High recall = few false negatives.)
- **F1** — the harmonic mean of precision and recall: `2 * P * R / (P + R)`.

For multi-class problems, you aggregate per-class F1 scores into an overall score. There are three common ways:

- **Macro-F1** — unweighted average of per-class F1 scores. Treats every class equally regardless of size. **This is the right metric for class-imbalanced settings where minority classes matter.**
- **Weighted-F1** — average weighted by class frequency. Dominated by majority class.
- **Micro-F1** — equivalent to accuracy in *single-label* multi-class classification (every example has exactly one true class, as in our positive / neutral / negative setup). The equivalence does not hold for multi-label problems, where an example can carry several labels at once. Less useful as a separate metric in the single-label case.

For AARSynth app review's three-way classification (positive / neutral / negative) with mild imbalance, **macro-F1 is the recommended primary metric.**

Confusion matrix

A square matrix where rows are true labels and columns are predicted labels. Diagonal = correct predictions; off-diagonal = errors. The pattern of off-diagonal errors reveals which classes the model confuses with which. For sentiment, the most informative confusion is usually neutral ↔ positive or neutral ↔ negative — the model has learned the polar classes well but struggles to identify true neutrality.

compute_metrics

`Trainer` calls a `compute_metrics` function during evaluation. The function receives an `EvalPrediction`-like object with two fields:

- `predictions` — the raw model output (logits — not probabilities, not class labels)
- `label_ids` — the gold labels

A typical implementation:

```python
import numpy as np
from sklearn.metrics import accuracy_score, f1_score

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=1)
    return {
        "accuracy": accuracy_score(labels, predictions),
        "macro_f1": f1_score(labels, predictions, average="macro"),
    }
```

Two important details:

- **`np.argmax` over axis 1** to convert logits to predicted class indices. Forgetting this is a common bug — you would feed raw logits into a metric that expects class indices, and the metric would crash or return nonsense.
- **`average="macro"`** for multi-class F1. The default is `"binary"` (which fails for multi-class) or `None` (which returns per-class array — also not what you want as a scalar).

Qualitative error analysis

Metrics tell you *how well* the model performs on average. They do not tell you *what kinds of mistakes* it makes. A 0.85 macro-F1 model could fail in many different ways — confusing neutral with positive, struggling with negation, misclassifying short sentences, etc. The Lab and Integration require you to look at specific misclassified examples and characterize the model's failure modes. This is the move from "I trained a model" to "I understand my model."

---

10. Compute Reality and Engineering Trade-offs

Fine-tuning is a knob-balancing act. Five knobs interact:

Knob	Lower value	Higher value
Model size	Faster, less memory, lower accuracy	Slower, more memory, higher accuracy
Dataset size	Faster to train, weaker model	Slower to train, stronger model
Number of epochs	Faster, may underfit	Slower, may overfit
Sequence length (`max_length`)	Faster (quadratic in length for attention)	Captures longer context
Batch size	Lower memory, more variance per step	Higher memory, smoother gradients

For CPU fine-tuning of DistilBERT on AARSynth app reviews, the canonical knob settings — and why:

- **Model:** DistilBERT (~66M params). Small enough that a full epoch on the curated ~7,500-example training split takes ~30–45 minutes on CPU.
- **Dataset:** ~7,500 labeled app reviews across 9 apps × 3 sentiment classes (curated subset of AARSynth). Large enough to fine-tune meaningfully; small enough that training fits in a laptop session.
- **Epochs:** 2. One epoch lets the model see every example once; two confirms learning. Three or four often help marginally but can overfit on a small dataset.
- **`max_length`:** 128. Most AARSynth app reviews sentences are well under 128 tokens; trim handles outliers.
- **Batch size:** 8. Fits comfortably in 4–6 GB of RAM with DistilBERT.

Total expected training time: **15–60–90 minutes on CPU**, depending on machine.

Memory peak: **~4 GB**. The course minimum laptop spec (8 GB) is sufficient if you close other heavy applications during training.

When fine-tuning is worth the cost

A real engineering question: when is it worth fine-tuning a custom model vs. using a public pre-trained model off the shelf?

Fine-tune when:

- You have labeled data in your domain (the labels exist, even if the dataset is small).
- A public pre-trained model on your task does not exist or is dramatically worse than what you need.
- The cost of mislabeling each example is meaningful (you care about quality more than convenience).
- Latency matters: a fine-tuned smaller model can outperform prompting a much larger model in production.

Use a pre-trained model off the shelf when:

- You don't have labels.
- A pre-trained model already exists for your task and is "good enough."
- You are prototyping and want to validate the task before investing in labeling.

The Integration Task this week applies the lab's fine-tuned classifier to a *different domain* (tech / entertainment news) than it was trained on (app reviews) and asks you to characterize how it fails. That gap — between "does well on data shaped like the training data" and "does well on real data your business cares about" — is the central question of applied ML.

Week B will revisit this trade-off explicitly: you will use *pre-trained* (not fine-tuned) models for question answering and summarization and write a comparative analysis of when fine-tuning is worth the cost.

Key Terms

- **Encoder, decoder, encoder–decoder** — the three transformer architectural families
- **Pre-training** — training a transformer on a large general corpus with a self-supervised objective (masked LM for encoders, causal LM for decoders)
- **Fine-tuning** — continuing training on a smaller task-specific labeled dataset to specialize the model
- **Transfer learning** — the principle that knowledge learned on one task (pre-training) transfers to another task (fine-tuning)
- **Backbone** — the pre-trained transformer weights, shared across tasks
- **Task-specific head** — a small neural network attached to the backbone, randomly initialized, that produces task-shaped outputs
- **`AutoModelForSequenceClassification`** — Hugging Face class that loads a backbone + classification head with output dim `num_labels`
- **`Trainer`** — Hugging Face class that wraps the training loop
- **`TrainingArguments`** — configuration object for `Trainer`
- **`DataCollatorWithPadding`** — pads each batch to the longest sequence in the batch (dynamic padding)
- **`compute_metrics`** — function passed to `Trainer` that converts model predictions and labels into a metric dict
- **DistilBERT, BERT, RoBERTa, DeBERTa** — BERT-family encoder models, varying in size and training recipe
- **Causal language modeling (CLM)** — the decoder pre-training and fine-tuning objective: predict next token
- **Masked language modeling (MLM)** — the encoder pre-training objective: predict hidden tokens
- **Parameter-efficient fine-tuning (PEFT)** — techniques (LoRA, adapters) that update only a small portion of weights
- **`datasets` library** — Hugging Face library for dataset loading, splitting, and tokenization at scale
- **`set_format("torch")`** — configures a Dataset to return PyTorch tensors
- **Attention mask** — binary mask marking real tokens (1) vs. pad tokens (0)
- **Truncation** — cutting sequences longer than `max_length` to fit the model's input limit
- **Logits** — raw, pre-softmax model outputs; `np.argmax` converts logits to class indices
- **Macro-F1** — unweighted average of per-class F1; the recommended single-number summary for class-imbalanced multi-class problems
- **Hugging Face Hub** — public model registry; pushing your fine-tuned model lets others (including your own future code) load it via `from_pretrained`
- **`from_pretrained` / `push_to_hub`** — load from / push to the Hub

Summary

Modern NLP rests on the pre-train + fine-tune paradigm. A pre-trained transformer is a general-purpose language model; fine-tuning specializes it to a task by continuing training on a small labeled dataset. The architecture (encoder, decoder, encoder–decoder) determines the natural shape of the task; the head determines the specific task within a family. For sequence classification — this Lab's task — you load `AutoModelForSequenceClassification` with the right `num_labels`, prepare a tokenized `DatasetDict` with `datasets`, configure `TrainingArguments` and `compute_metrics`, and run `Trainer.train()`. After training, evaluate with macro-F1 and a confusion matrix, and analyze qualitative errors. CPU fine-tuning is feasible only for small encoders (DistilBERT) on small datasets with small batches and few epochs — and that constraint is a real engineering trade-off, not a course shortcut.

What's Next

Tonight's drill: prepare a labeled dataset and configure `Trainer` (no actual training — that's the Lab).

Monday's lab: fine-tune DistilBERT on AARSynth app reviews, evaluate, push your model to Hugging Face Hub, write an evaluation report.

Wednesday's integration: apply your fine-tuned classifier (loaded from your Hugging Face Hub) to a different domain (tech / entertainment news) and analyze where the model breaks.

Week B: pre-trained encoder–decoder models for summarization, extractive question answering, and a comparative evaluation report of when fine-tuning is worth the cost.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.